

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 1
з дисципліни “Професійний Python”
на тему “Організація професійного Python-проєкту та Git”

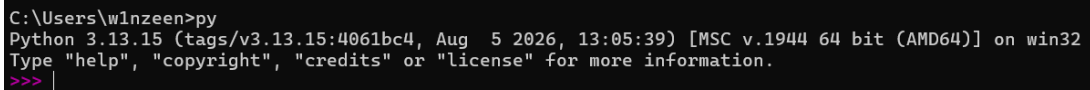
Виконав
студент 3 курсу
групи ФЕП-33
Рак Богдан Ігорович

Перевішив
Професор кафедри ОІТ
Свелеба Сергій Андрійович

Львів – 2026

Мета: Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

1. Встановити Python.



```
C:\Users\wInzeen>py
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

Рис. 1. Перевірка встановленої версії Python у командному рядку Windows.

Команда ru запускає інтерактивний інтерпретатор і виводить версію Python 3.13.15, що підтверджує коректне встановлення інтерпретатора в системі.

2. Створити каталог проєкту.

Створив теку для проєкту

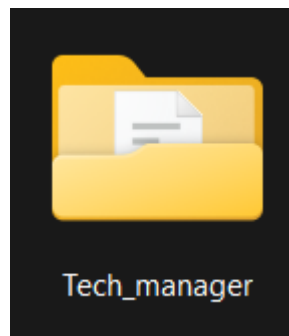
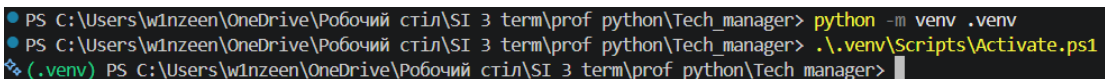


Рис. 2. Створена на робочому столі порожня тека Tech_manager, яка слугуватиме кореневим каталогом майбутнього Python-проєкту.

3. Створити .venv, 4. Активувати virtual environment.

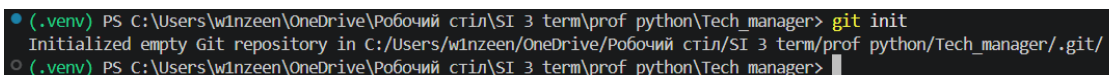


```
PS C:\Users\wInzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> python -m venv .venv
PS C:\Users\wInzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\wInzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> |
```

Рис. 3. Термінал PowerShell у теці проєкту: командою python -m venv .venv створено ізольоване віртуальне середовище, а командою .venv\Scripts\Activate.ps1 його активовано — про це свідчить префікс (.venv) перед шляхом у рядку запрошення.

Створив віртуальне середовище та активував його.

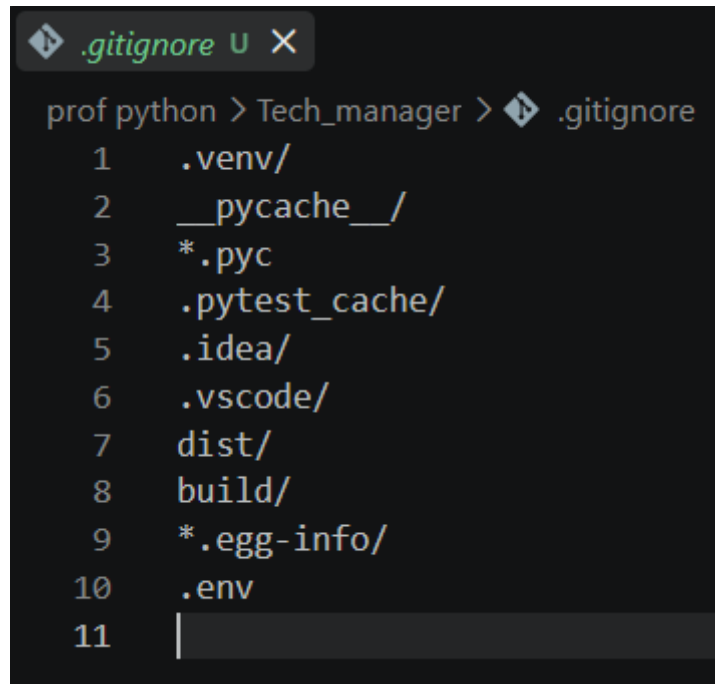
5. Ініціалізувати Git repository.



```
(.venv) PS C:\Users\wInzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> git init
Initialized empty Git repository in C:/Users/wInzeen/OneDrive/Робочий стіл/SI 3 term/prof python/Tech_manager/.git/
(.venv) PS C:\Users\wInzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> |
```

Рис. 4. Виконання команди `git init` у корені проєкту. Git повідомляє про створення порожнього репозиторію (*Initialized empty Git repository*) у прихованому каталозі `.git` всередині теки `Tech_manager`.

6. Створити `.gitignore`.



```
.gitignore U X
prof python > Tech_manager > .gitignore
1 .venv/
2 __pycache__/
3 *.pyc
4 .pytest_cache/
5 .idea/
6 .vscode/
7 dist/
8 build/
9 *.egg-info/
10 .env
11 |
```

Рис. 5. Вміст файлу `.gitignore` у редакторі коду. У файл додано шаблони службових файлів і каталогів, які Git повинен ігнорувати: `.venv/`, `__pycache__/`, `*.pyc`, `.pytest_cache/`, `.idea/`, `.vscode/`, `dist/`, `build/`, `*.egg-info/` та `.env`.

7. Створити `src-layout`.

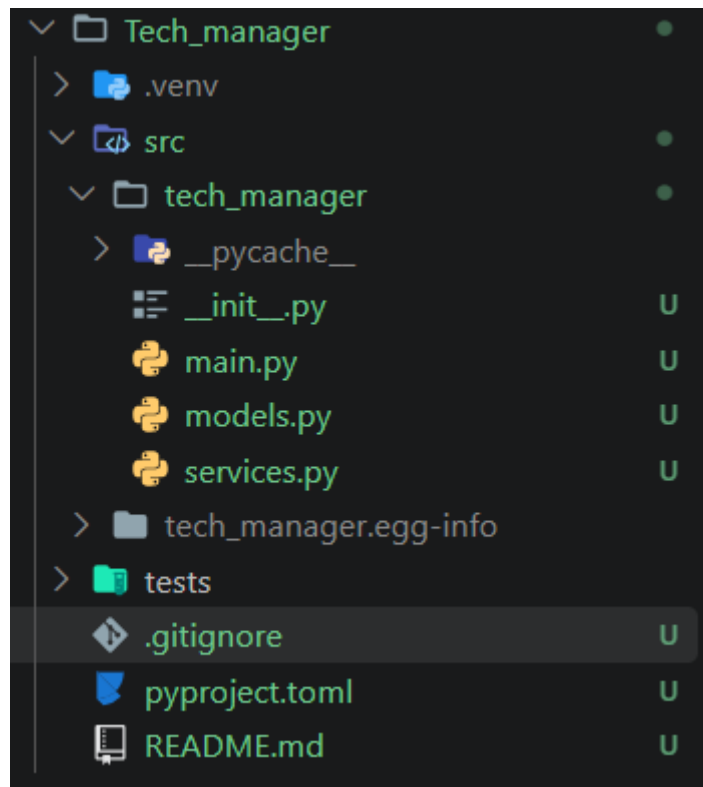


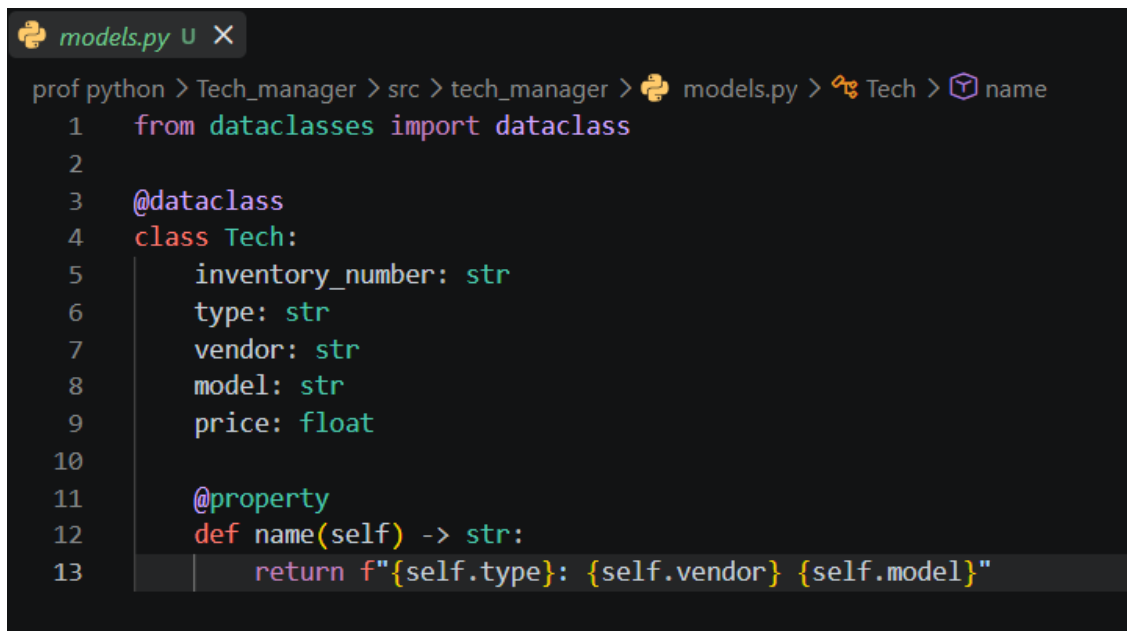
Рис. 6. Структура проєкту за принципом *src-layout* у бічній панелі VS Code: пакет *tech_manager* розміщено всередині каталогу *src*, а файли *__init__.py*, *main.py*, *models.py* та *services.py* — усередині пакета; окремо видно каталог *tests*, файли *.gitignore*, *pyproject.toml* та *README.md* у корені проєкту.

8. Створити package.

```
(.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech manager> python -m pip install -e .
Obtaining file:///C:/Users/winzeen/OneDrive/Робочий стіл\SI 3 term\prof python\Tech manager
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: tech-manager
Building editable for tech-manager (pyproject.toml) ... done
Created wheel for tech-manager: filename=tech_manager-0.1.0-0.editable-py3-none-any.whl size=1604 sha256=679853bcf2cbb3d10216176901d0917e8b1ee8357d23364b133280949691c73e
Stored in directory: C:\Users\winzeen\AppData\Local\Temp\pip-ephem-wheel-cache-ee6aeb3u\wheels\8c\24\d0\d35d8f318125127c82fe56d5fdae0c8d95f1dfa
f8582fed618
Successfully built tech-manager
Installing collected packages: tech-manager
Attempting uninstall: tech-manager
Found existing installation: tech-manager 0.1.0
Uninstalling tech-manager-0.1.0:
Successfully uninstalled tech-manager-0.1.0
Successfully installed tech-manager-0.1.0
(.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech manager>
```

Рис. 7. Виконання команди *python -m pip install -e .* для встановлення пакета *tech-manager* у режимі розробки (*editable mode*). У виводі показано підготовку метаданих із *pyproject.toml*, збирання *editable-wheel* файлу та успішне встановлення пакета *tech-manager 0.1.0*.

9. Створити models.



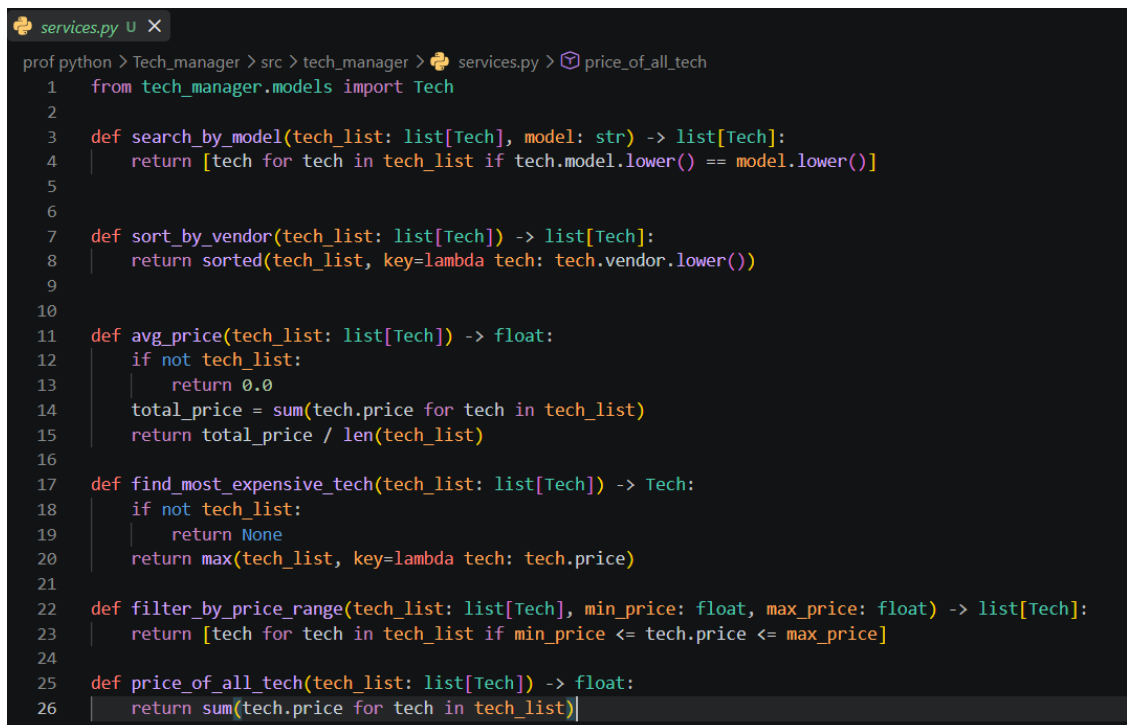
```

models.py U X
prof python > Tech_manager > src > tech_manager > models.py > Tech > name
1  from dataclasses import dataclass
2
3  @dataclass
4  class Tech:
5      inventory_number: str
6      type: str
7      vendor: str
8      model: str
9      price: float
10
11     @property
12     def name(self) -> str:
13         return f"{self.type}: {self.vendor} {self.model}"

```

Рис. 8. Вміст файлу `models.py`: за допомогою декоратора `@dataclass` описано клас `Tech` із полями `inventory_number`, `type`, `vendor`, `model` і `price`, а також властивість (property) `name`, яка формує зручне текстове представлення пристрою у форматі 'тип: виробник модель'.

10. Створити `services.py`.



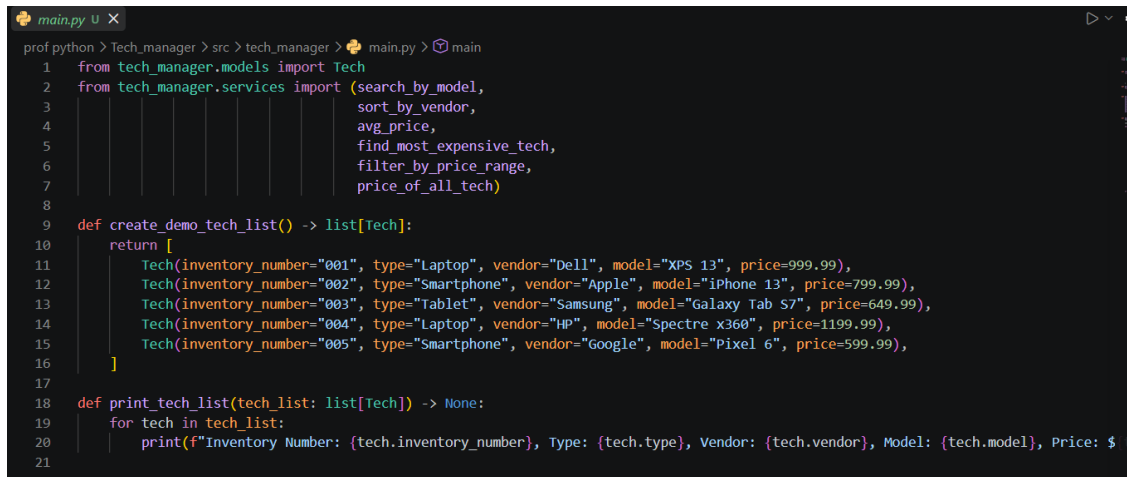
```

services.py U X
prof python > Tech_manager > src > tech_manager > services.py > price_of_all_tech
1  from tech_manager.models import Tech
2
3  def search_by_model(tech_list: list[Tech], model: str) -> list[Tech]:
4      return [tech for tech in tech_list if tech.model.lower() == model.lower()]
5
6
7  def sort_by_vendor(tech_list: list[Tech]) -> list[Tech]:
8      return sorted(tech_list, key=lambda tech: tech.vendor.lower())
9
10
11 def avg_price(tech_list: list[Tech]) -> float:
12     if not tech_list:
13         return 0.0
14     total_price = sum(tech.price for tech in tech_list)
15     return total_price / len(tech_list)
16
17 def find_most_expensive_tech(tech_list: list[Tech]) -> Tech:
18     if not tech_list:
19         return None
20     return max(tech_list, key=lambda tech: tech.price)
21
22 def filter_by_price_range(tech_list: list[Tech], min_price: float, max_price: float) -> list[Tech]:
23     return [tech for tech in tech_list if min_price <= tech.price <= max_price]
24
25 def price_of_all_tech(tech_list: list[Tech]) -> float:
26     return sum(tech.price for tech in tech_list)

```

Рис. 9. Вміст файлу `services.py`: реалізовано функції бізнес-логіки для роботи зі списком техніки — `search_by_model` (пошук за моделлю), `sort_by_vendor` (сортування за виробником), `avg_price` (середня вартість), `find_most_expensive_tech` (найдорожчий пристрій), `filter_by_price_range` (фільтрація за діапазоном цін) та `price_of_all_tech` (загальна вартість).

11. Створити `main.py`.



```

main.py X
prof python > Tech_manager > src > tech_manager > main.py > main
1 from tech_manager.models import Tech
2 from tech_manager.services import (search_by_model,
3                                   sort_by_vendor,
4                                   avg_price,
5                                   find_most_expensive_tech,
6                                   filter_by_price_range,
7                                   price_of_all_tech)
8
9 def create_demo_tech_list() -> list[Tech]:
10     return [
11         Tech(inventory_number="001", type="Laptop", vendor="Dell", model="XPS 13", price=999.99),
12         Tech(inventory_number="002", type="Smartphone", vendor="Apple", model="iPhone 13", price=799.99),
13         Tech(inventory_number="003", type="Tablet", vendor="Samsung", model="Galaxy Tab S7", price=649.99),
14         Tech(inventory_number="004", type="Laptop", vendor="HP", model="Spectre x360", price=1199.99),
15         Tech(inventory_number="005", type="Smartphone", vendor="Google", model="Pixel 6", price=599.99),
16     ]
17
18 def print_tech_list(tech_list: list[Tech]) -> None:
19     for tech in tech_list:
20         print(f"Inventory Number: {tech.inventory_number}, Type: {tech.type}, Vendor: {tech.vendor}, Model: {tech.model}, Price: $
21
22

```

Рис. 10. Вміст файлу main.py: функція `create_demo_tech_list()` створює демонстраційний список із п'яти об'єктів `Tech` (ноутбуки, смартфони, планшети різних виробників), а функція `print_tech_list()` виводить у консоль інформацію про кожен пристрій зі списку.

12. Реалізувати свій індивідуальний варіант.

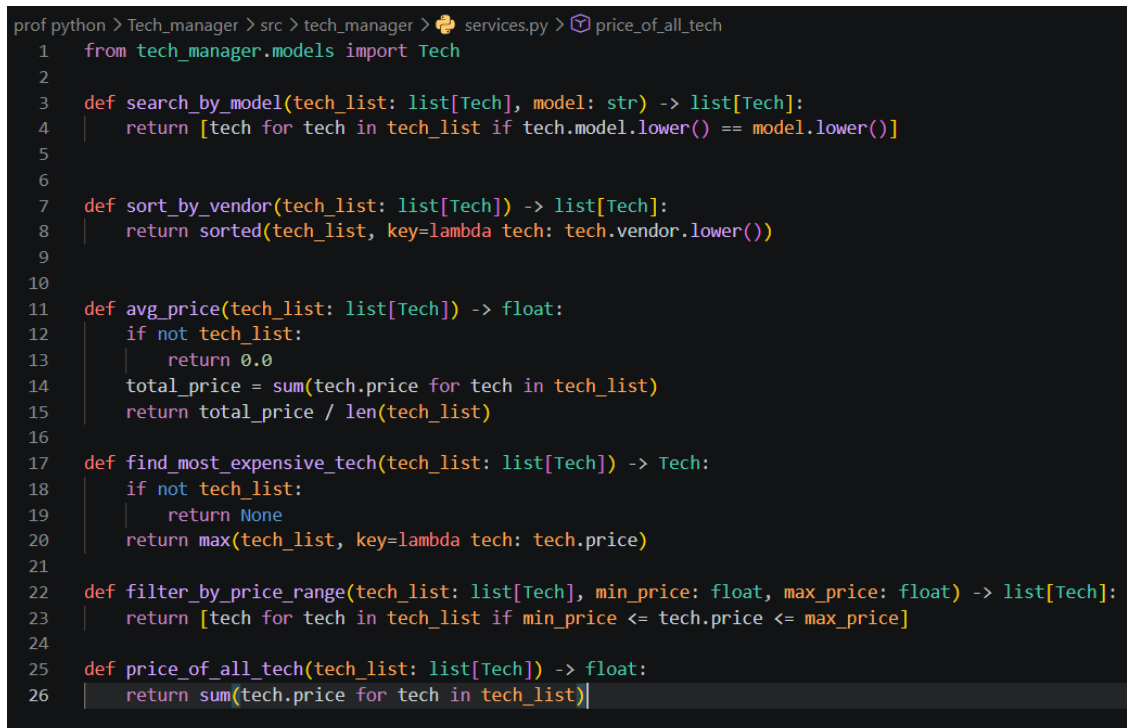
Оскільки я 13 в списку, я реалізував 13 варіант.

Варіант 13. Система обліку комп'ютерної техніки

Для пристрою: інвентарний номер; тип; виробник; модель; вартість.

Реалізувати: пошук; фільтрацію за виробником; середню вартість; найдорожчий пристрій; загальну вартість обладнання.

13. Додати type hints.



```

prof python > Tech_manager > src > tech_manager > services.py > price_of_all_tech
1 from tech_manager.models import Tech
2
3 def search_by_model(tech_list: list[Tech], model: str) -> list[Tech]:
4     return [tech for tech in tech_list if tech.model.lower() == model.lower()]
5
6
7 def sort_by_vendor(tech_list: list[Tech]) -> list[Tech]:
8     return sorted(tech_list, key=lambda tech: tech.vendor.lower())
9
10
11 def avg_price(tech_list: list[Tech]) -> float:
12     if not tech_list:
13         return 0.0
14     total_price = sum(tech.price for tech in tech_list)
15     return total_price / len(tech_list)
16
17 def find_most_expensive_tech(tech_list: list[Tech]) -> Tech:
18     if not tech_list:
19         return None
20     return max(tech_list, key=lambda tech: tech.price)
21
22 def filter_by_price_range(tech_list: list[Tech], min_price: float, max_price: float) -> list[Tech]:
23     return [tech for tech in tech_list if min_price <= tech.price <= max_price]
24
25 def price_of_all_tech(tech_list: list[Tech]) -> float:
26     return sum(tech.price for tech in tech_list)

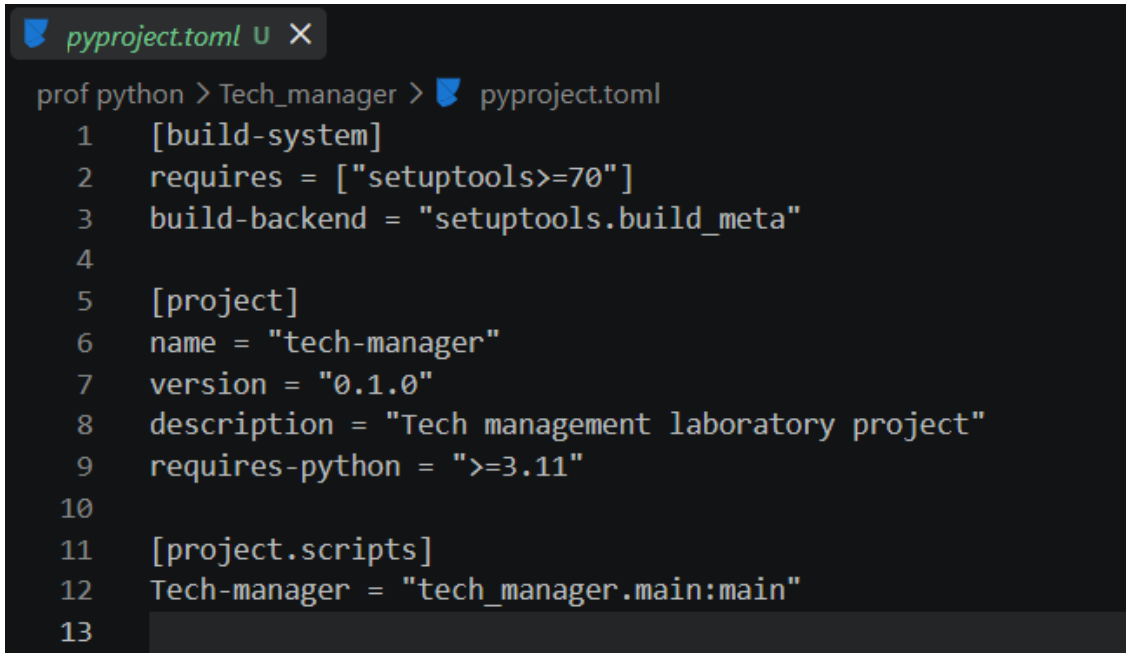
```

Рис. 11. Файл `services.py` після додавання анотацій `типів` (`type hints`) до всіх функцій: явно вказано типи параметрів (наприклад, `list[Tech]`, `str`, `float`) і

типи значень, що повертаються (`list[Tech]`, `float`, `Tech`), що підвищує читабельність коду й дозволяє статичним аналізаторам перевіряти коректність типів.

Додав type hints для функцій.

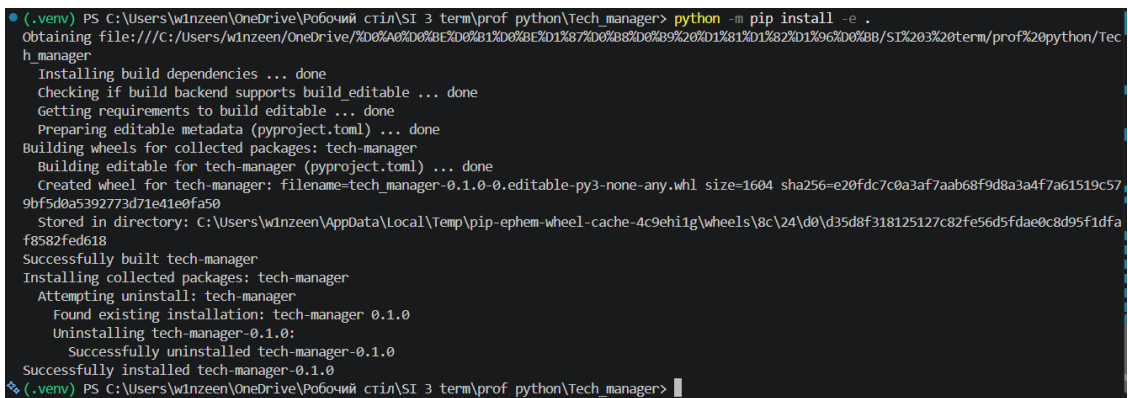
14. Створити `pyproject.toml`.



```
pyproject.toml
prof python > Tech_manager > pyproject.toml
1  [build-system]
2  requires = ["setuptools>=70"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "tech-manager"
7  version = "0.1.0"
8  description = "Tech management laboratory project"
9  requires-python = ">=3.11"
10
11 [project.scripts]
12 Tech-manager = "tech_manager.main:main"
13
```

Рис. 12. Вміст файлу `pyproject.toml`: у секції `[build-system]` вказано `setuptools` як інструмент збирання, а в секції `[project]` — назву пакета `tech-manager`, версію `0.1.0`, опис, мінімальну версію Python (`>=3.11`) та точку входу консольної команди `tech-manager` у секції `[project.scripts]`.

15. Встановити package у `editable mode`.



```
(.venv) PS C:\Users\w1nzen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> python -m pip install -e .
Obtaining file:///C:/Users/w1nzen/OneDrive/Робочий стіл\SI 3 term\prof python\Tech_manager
Installing build dependencies ... done
Checking if build backend supports build editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: tech-manager
Building editable for tech-manager (pyproject.toml) ... done
Created wheel for tech-manager: filename=tech_manager-0.1.0-0.editable-py3-none-any.whl size=1604 sha256=e20fdc7c0a3af7aab68f9d8a3a4f7a61519c579bf5d0a5392773d71e41e0fa50
Stored in directory: C:\Users\w1nzen\AppData\Local\Temp\pip-ephem-wheel-cache-4c9ehi1g\wheels\8c\24\d0\d35d8f318125127c82fe56d5fdae0c8d95f1dfa
f8582fed618
Successfully built tech-manager
Installing collected packages: tech-manager
Attempting uninstall: tech-manager
Found existing installation: tech-manager 0.1.0
Uninstalling tech-manager-0.1.0:
Successfully uninstalled tech-manager-0.1.0
Successfully installed tech-manager-0.1.0
(.venv) PS C:\Users\w1nzen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager>
```

Рис. 13. Повторне виконання команди `python -m pip install -e .` після оновлення `pyproject.toml`. Пакет `tech-manager` перевстановлюється в `editable-режимі` з урахуванням нових метаданих та доданої консольної команди.

16. Перевірити запуск.

```

(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> tech-manager

Tech Manager Menu:
1. View all tech items
2. Search by model
3. Sort by vendor
4. Calculate average price
5. Find the most expensive tech item
6. Filter by price range
7. Calculate total price of all tech items
8. Exit
Enter your choice: 1

All Tech Items:
Inventory Number: 001, Type: Laptop, Vendor: Dell, Model: XPS 13, Price: $999.99
Inventory Number: 002, Type: Smartphone, Vendor: Apple, Model: iPhone 13, Price: $799.99
Inventory Number: 003, Type: Tablet, Vendor: Samsung, Model: Galaxy Tab S7, Price: $649.99
Inventory Number: 004, Type: Laptop, Vendor: HP, Model: Spectre x360, Price: $1199.99
Inventory Number: 005, Type: Smartphone, Vendor: Google, Model: Pixel 6, Price: $599.99

Tech Manager Menu:
1. View all tech items
2. Search by model
3. Sort by vendor
4. Calculate average price
5. Find the most expensive tech item
6. Filter by price range
7. Calculate total price of all tech items
8. Exit
Enter your choice: 2
Enter the model to search for: Galaxy Tab S7

Search Results:
Inventory Number: 003, Type: Tablet, Vendor: Samsung, Model: Galaxy Tab S7, Price: $649.99

```

Рис. 14. Результат запуску консольної команди `tech-manager` у терміналі: відображено текстове меню програми з 8 пунктами, приклад перегляду всіх пристроїв (пункт меню 1) та приклад пошуку пристрою за моделлю 'Galaxy Tab S7' (пункт меню 2), який успішно повертає відповідний запис.

17. Створити README.md.

```
README.md X
prof python > Tech_manager > README.md > abc # Tech Manager > abc ## Description
1  # Tech Manager
2
3  Laboratory project for the course
4  "Professional Python".
5
6  ## Description
7
8  Console application for managing technic.
9
10 ## Requirements
11
12 Python 3.11+
13
14 ## Installation
15
16 python -m venv .venv
17
18 Activate the virtual environment.
19
20 python -m pip install -e .
21
22 ## Run
23
24 python -m tech_manager.main
25
26 or
27
28 tech-manager
29
30 ## Project structure
31
32 src/tech_manager/models.py
33 Data models.
34
35 src/tech_manager/services.py
36 Business logic.
```

Рис. 15. Вміст файлу README.md у редакторі: документ містить назву проєкту, опис, вимоги до версії Python, покрокову інструкцію зі встановлення (створення venv, встановлення пакета) та запуску програми, а також опис структури проєкту (models.py, services.py).

18. Зробити не менше трьох Git commits.

```
(.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> git log --oneline
5ca2695 (HEAD -> main, origin/main) Created .toml file
513968d Created .gitignore file
59d0217 Created README file
f6bfc80 Created init file
3257f66 Created services file
d12f10f Created models file
f432e2e Created main file
(.venv) PS C:\Users\winzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager>
```

Рис. 16. Вивід команди git log --oneline: історія комітів проєкту містить сім записів (від Created main file до Created .toml file), тобто вимогу зробити щонайменше три коміти виконано з запасом.

19. Перевірити git status.

```
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
(.venv) PS C:\Users\w1nzeen\OneDrive\Робочий стіл\SI 3 term\prof python\Tech_manager> █
```

Рис. 17. Вивід команди `git status`: поточна гілка `main` синхронізована з віддаленою гілкою `origin/main`, робоче дерево чисте (`nothing to commit, working tree clean`) — усі зміни вже закомічені.

Висновок: Виконуючи цю лабораторну роботу, можна ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом `src-layout`, працювати з модулями та пакетами Python, створювати файл `pyproject.toml`, використовувати Git для контролю версій та оформлювати базову документацію проєкту.